

K-Means-Clustering Accuracy vs. Random Selection Accuracy

Danny Quang

Abstract

This document explores my idea for prototype selection in nearest neighbor classification. Specifically, I explore the k-means clustering prototype selection, which groups data based on their similarity.

We also try random selection, where we select random points as our centroids, and use 1NN to classify the points, or just assign them to the label of the closest neighbor.

Random selection is a baseline method, and k-means aims to increase accuracy by choosing better points for 1NN classification.

1 Description

In my experiment, I implement and run k-means neighbors on the MNIST dataset in hopes to improve the "centroid" selection.

First, I run a baseline algorithm where I pick M random points to serve as "centroids" so that points can classify as the same label as the closest randomly chosen centroid. This works by choosing a random subset of the points.

Then, I implement and run k-means neighbors. This groups similar points into groups and chooses "centroids" that are closer to the average of the group's closest points. In doing so, it minimizes the sum of distances between the data points and their cluster centroids. This works by first picking some centroids, or central points, and assign each data points to its closest centroid. Then, we find the average point of each cluster, and update the centroid position. This process repeats until it converges.

2 Pseudocode

Algorithm: Random Selection

INPUT: Training data X, training labels y, and M prototypes

OUTPUT: Selected prototypes and their labels of size M

1. Randomly sample M indices without replacement.
2. Return corresponding data points and labels for the random centroids.

Algorithm: K-Means Clustering

INPUT: Training data X, training labels y, and M prototypes

OUTPUT: Selected prototypes and their labels of size M

1. Get the unique number labels and count the frequency of each number to fairly distribute the centroids to each number.
numberLabels = unique(trainLabels)
samplesPerNumber = count the frequency of images per number
amtClustersPerNumber = ceil (M *samplesPerNumber / # yTrain labels)
2. Due to rounding error, there may be some extra or missing clusters. Make sure to redistribute them.
If sum(amtClustersPerNumber) < M:
- give another cluster to most common number or take one from the least common number
3. For each number, apply k-means with its allocated clusters:
- Initialize n cluster centers, or "centroids"
- Iterate until convergence:
- - Assign points to closest centroid
- - Update centroid as the average of the points
4. For each centroid, find the closest data point to it:
- Go thru each point and rank by euclidian distance.
- take the closest point
- save the point's index
5. Return the points closest to all of the centroids

3 Experiment Results

I tested the two methods on the MNIST dataset with 60,000 training images with labels, and 10,000 test images with labels.

The k-Means selections algorithm is not random, so it was sufficient to compute each accuracy value for each M once. However, the random selection algorithm is random, so I ran it for 5 runs and averaged the accuracy values and found the standard deviation. With those values, I used a confidence interval with 95% confidence, or 1.96 standard deviations, to capture 95% of the area under a normal curve to compute the experiment run intervals.

$$CI = (\bar{x}) \pm (1.96 * std_deviation \div \sqrt{5})$$

At 1,000 prototypes, we can see a more pronounced difference in accuracy, with k-Means achieving an average of 92.52% accuracy while randomly selected prototypes only achieve an average of 88.53% accuracy. This small error in random selection show that this is near constant performance, and so it is evident that k-Means is an effective strategy for the MNIST dataset. My results are as follows:

Accuracy			
Approach	M:1000	M: 5000	M:10,000
k-Means-Clustering	.9252	.9492	.956
Random Selection	.8853 $\pm$ 0.0049	.9361 $\pm$.0049	.9606 $\pm$ 0

Table 1: Classification accuracy with Random Selection and k-Means-Clustering with M=1000, 5000, 10,000 prototypes. Error shows 95% confidence intervals for random selection.

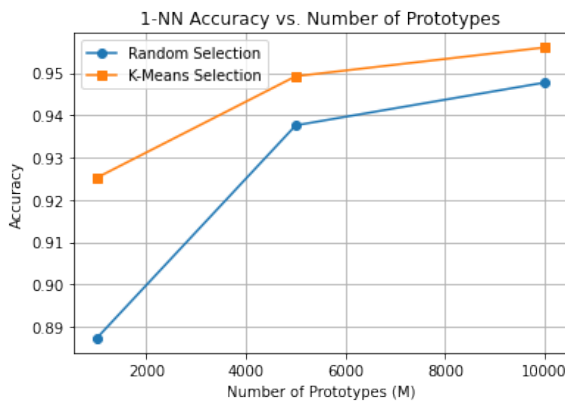


Figure 1: Line plot of classification accuracy with

Random Selection and k-Means-Clustering with M=1000, 5000, 10,000 prototypes.

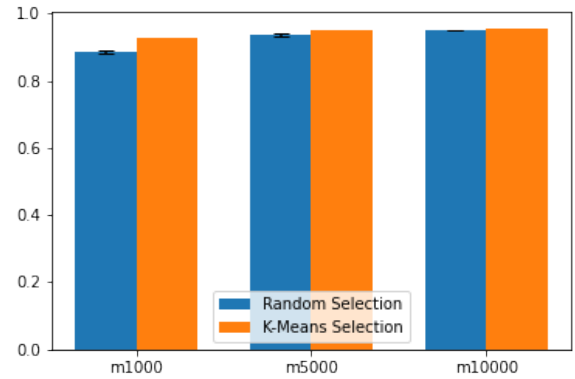


Figure 2: Full bar plot of classification accuracy with Random Selection and k-Means-Clustering with M=1000, 5000, 10,000 prototypes.

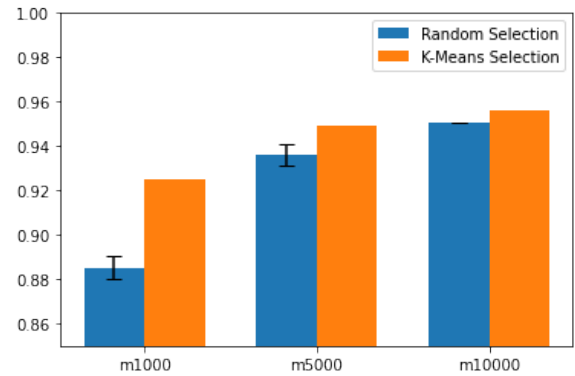


Figure 3: Zoomed in, or chopped, bar chart of classification accuracy with Random Selection and k-Means-Clustering with M=1000, 5000, 10,000 prototypes.

4 Critical Evaluation

As the results show, k-Means-Clustering shows an improvement over random selection. At 10,000 prototypes, there is a smaller but still noticeable improvement, where k-Means achieved a 95.6% accuracy while randomly selected prototypes surprisingly achieved 95.06% accuracy. We can attribute this to the sheer amount of possible centroids to choose from.

There is further scope for improvement, namely that that algorithm is further optimized in speed with libraries like KNearestNeighbors. I want to try using neural networks to achieve higher accuracies.